

HW1 – CS 671

Yash Dagade

September 19, 2025

(a) No Regularization

1. **Why optimizing plain 0–1 loss is bad.** Because without a sparsity/complexity penalty a tree can grow arbitrarily deep to perfectly memorize the training set (zero training error) and badly overfit. In this case we would get really deep trees that may not generalize very well at all and would have just learning to memorize the training data and that's no good.
2. **Deepest valid tree.** Along any root-to-leaf path you can test each feature at most once, so I can have at most k splits. Since depth counts nodes (leaf included) and a tree with one leaf has depth 1, the deepest valid tree has depth

$$\boxed{k + 1}.$$

3. **Number of valid optimal trees for Table 1.** Dataset: $(1, 0) \rightarrow 0$ (twice), $(0, 1) \rightarrow 1$ (once). Zero training error is achievable.

- Depth-2 stumps:
 - (a) Root split on x_1 : $x_1=1 \mapsto 0$, $x_1=0 \mapsto 1$.
 - (b) Root split on x_2 : $x_2=1 \mapsto 1$, $x_2=0 \mapsto 0$. $\Rightarrow$ 2 trees.
- Depth-3 extensions (root x_1 , mirror for x_2):
 - Split only one child on x_2 : 2 choices of child $\times$ 2 free labelings = 4 trees.
 - Split both children on x_2 : 2 leaves fixed, 2 free $\Rightarrow 2^2 = 4$ trees. $\Rightarrow$ 8 trees per root. Since root can be x_1 or x_2 , total 16.

Total valid optimal trees:

$$\boxed{18}.$$

I drew all 18 trees. Please refer to the appendix for a more detailed illustration of each of the 18 unique trees. :D

4. **Unique logical expressions among these trees.** Constraints: $f(1, 0) = 0$, $f(0, 1) = 1$. Unseen points $(0, 0)$ and $(1, 1)$ are free, thus can be anything. That yields $2 \times 2 = 4$ distinct Boolean functions that produce the right answer for our constraints.

$$\begin{aligned} (0, 0) \rightarrow 0, (1, 1) \rightarrow 0 : f &= \neg x_1 \wedge x_2, \\ (0, 0) \rightarrow 0, (1, 1) \rightarrow 1 : f &= x_2, \\ (0, 0) \rightarrow 1, (1, 1) \rightarrow 0 : f &= \neg x_1, \\ (0, 0) \rightarrow 1, (1, 1) \rightarrow 1 : f &= \neg x_1 \vee x_2. \end{aligned}$$

So the number is

$$\boxed{4}.$$

(b) Soft Regularization

We define

$$R(f, X, y | \lambda) = L(f, X, y) + \lambda s(f),$$

where $s(f)$ is the number of leaves.

1. Largest possible R for one-leaf tree.

A one-leaf tree (constant predictor) always predicts the majority class, so its error is at most $L = \frac{1}{2}$. Since it has $s(f) = 1$ leaf, we obtain

$$R(f, X, y | \lambda) = L(f, X, y) + \lambda s(f) \leq \frac{1}{2} + \lambda.$$

Formally, this inequality is the correct upper bound for any $\lambda \geq 0$. However, if $\lambda > \frac{1}{2}$, the regularization penalty dominates: adding more leaves can never reduce the error by more than 1, so the trivial one-leaf tree will always be optimal. In that sense, while λ is not mathematically capped at $\frac{1}{2}$, values much larger than $\frac{1}{2}$ effectively force the trivial solution. Thus we can say

$$R \leq \frac{1}{2} + \lambda,$$

or, if we restrict attention to the most meaningful range of λ ,

$$R \leq \frac{1}{2} + \frac{1}{2} = 1.$$

2. **Accuracy needed for deeper prefix-refinement.** Refining f_d into f_{d+k} adds at least k leaves, so $s(f_{d+k}) \geq s(f_d) + k$. For R to improve:

$$L(f_{d+k}) + \lambda(s(f_d) + k) < L(f_d) + \lambda s(f_d),$$

hence

$$L(f_{d+k}) \leq L(f_d) - \lambda k.$$

So the deeper tree must reduce misclassification by at least

$$\boxed{\lambda k}.$$

3. **Depth guarantee for suboptimality.** Any depth- d tree has at least $s \geq d$ leaves, hence $R \geq \lambda d$. A depth-1 tree has $R \leq \frac{1}{2} + \lambda$. Therefore any depth d with

$$\lambda d > \frac{1}{2} + \lambda \iff d > 1 + \frac{1}{2\lambda}$$

is always worse than some one-leaf tree. Thus the smallest such depth is

$$\boxed{d^*(\lambda) = \left\lfloor 1 + \frac{1}{2\lambda} \right\rfloor + 1}.$$

(If this exceeds the maximum feasible depth $k + 1$, the statement is vacuously true.)

4. **Number of optimal trees for Table 1**, $0 < \lambda < \frac{1}{2}$.

Candidate objectives:

- One-leaf majority (predict 0): $R = \frac{1}{3} + \lambda$.
- Zero-loss stumps (2 leaves): $R = 2\lambda$.
- Deeper zero-loss trees (3 or 4 leaves): $R = 3\lambda$ or 4λ (never optimal for $\lambda > 0$).

Compare 2λ vs $\frac{1}{3} + \lambda$:

- $0 < \lambda < \frac{1}{3}$: stumps win $\Rightarrow \boxed{2}$ optimal trees.
- $\lambda = \frac{1}{3}$: tie $\Rightarrow 2$ stumps + 1 one-leaf $\Rightarrow \boxed{3}$ optimal trees.
- $\frac{1}{3} < \lambda < \frac{1}{2}$: one-leaf wins $\Rightarrow \boxed{1}$ optimal tree.

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: Brian Mason and Cursor/GPT5 -- though I do want to emphasize that the code even if LLM generated was based on my reasoning and understanding of the problem and LLMs were simply used to help me write the code in a more efficient manner.

Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

All 18 trees printed out:

Dataset:

X Y Output:

1 & 0 & 0

0 & 1 & 1

1 & 0 & 0

We have two free inputs -

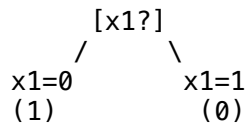
1 & 1

0 & 0

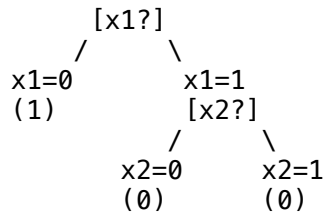
where the output could be either 1 or 0 :D

Trees with Root = x_1

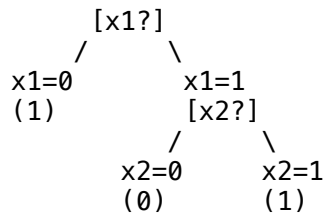
Tree1:



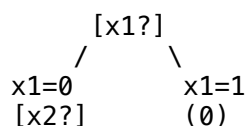
Tree 2 (free leaf=0)



Tree 3 (free leaf=1)



Tree 4 (free leaf=0)



```

  /      \
x2=0      x2=1
(0)      (1)

```

Tree 5 (free leaf=1)

```

      [x1?]
     /    \
  x1=0      x1=1
  [x2?]    (0)
   /    \
x2=0      x2=1
(1)      (1)

```

Tree 6 (0,0=0; 1,1=0)

```

      [x1?]
     /    \
  x1=0      x1=1
  [x2?]    [x2?]
   /    \   /    \
(0,0) (0,1) (1,0) (1,1)
(0)    (1)  (0)   (0)

```

Tree 7 (0,0=0; 1,1=1)

```

      [x1?]
     /    \
  x1=0      x1=1
  [x2?]    [x2?]
   /    \   /    \
(0,0) (0,1) (1,0) (1,1)
(0)    (1)  (0)   (1)

```

Tree 8 (0,0=1; 1,1=0)

```

      [x1?]
     /    \
  x1=0      x1=1
  [x2?]    [x2?]
   /    \   /    \
(0,0) (0,1) (1,0) (1,1)
(1)    (1)  (0)   (0)

```

Tree 9 (0,0=1; 1,1=1)

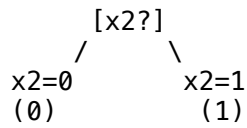
```

      [x1?]
     /    \
  x1=0      x1=1
  [x2?]    [x2?]
   /    \   /    \
(0,0) (0,1) (1,0) (1,1)
(1)    (1)  (0)   (1)

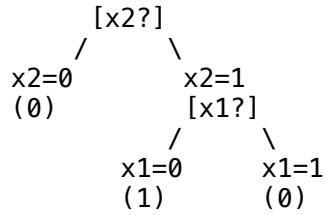
```

Trees with Root = \$x_2\$
Almost symmetric as before in structure!

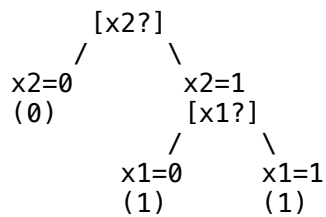
Tree 10



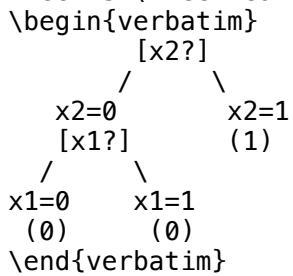
Tree 11 (free leaf=0)



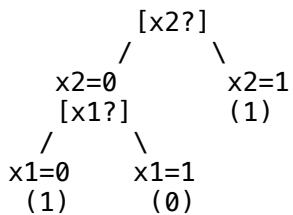
Tree 12 (free leaf=1)



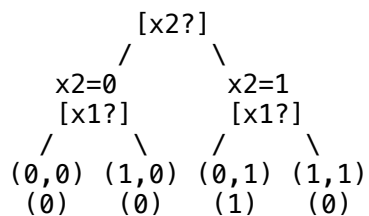
Tree 13 (free leaf=0)



Tree 14 (free leaf=1)

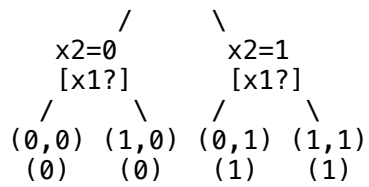


Tree 15 (0,0=0; 1,1=0)

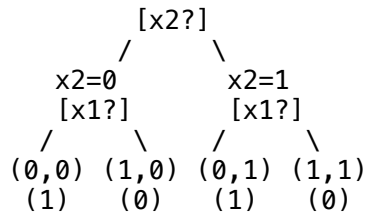


Tree 16 (0,0=0; 1,1=1)

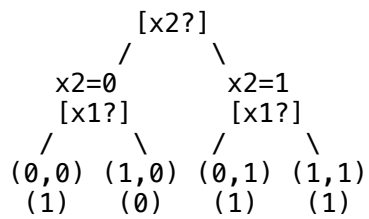




Tree 17 (0,0=1; 1,1=0)



Tree 18 (0,0=1; 1,1=1)



CS-671, Fall 2025 - HW 1, Q2

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: Brian Mason and Cursor/GPT5 -- though I do want to emphasize that the code even if LLM generated was based on my reasoning and understanding of the problem and LLMs were simply used to help me write the code in a more efficient manner. Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

StarterCode Setup

Importing Libraries

```
In [4]: import numpy as np
        from numpy.linalg import norm
        from sklearn.datasets import load_breast_cancer
        from sklearn.model_selection import train_test_split

        import matplotlib.pyplot as plt
```

Loading Dataset, Splitting normalizer into Train and Test sets

```
In [5]: bcd = load_breast_cancer()
        X = bcd.data
        y = bcd.target
        y = 1 - y

        # Split the dataset with a fixed random seed (random_state=42)
        X_train, X_test, y_train, y_test = train_test_split(
            X, y,
            test_size=0.2,
            random_state=42,
            stratify=y
        )

        print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

        # so it seems we have 455 samples in the training set and 114 in the test set
(455, 30) (114, 30) (455,) (114,)
```

we have to implement our own KNN classifier and our own cross validation algorithm - we are not allowed to use sklearn cross validation library.

It seems like the breast cancer dataset contains 30 features.

The scikit-learn breast cancer dataset is imbalanced, with approximately 63% benign and 37% malignant samples.

Q2 (a): Is accuracy or F1 more appropriate?

Accuracy can be misleading on imbalanced datasets because it is dominated by the majority class. Breast cancer data is moderately imbalanced, and false negatives (predicting benign when malignant) are particularly costly. The F1 score, being the harmonic mean of precision and recall, focuses on performance for the positive class and balances missed positives (FN) and false alarms (FP). Therefore, F1 is more appropriate here.

```
In [6]: # Clean, minimal, well-documented implementations
from typing import Callable, Tuple

# note used SciKit Learn MinMaxScaler to write safe normalizer that does not

class SafeNormalizer:
    """ See for normalizing, we can either normalize the columns or the rows
    Fits on training data; transforms with feature-wise min/max.
    Prevents leakage by computing stats only on the training set passed to fit
    """
    def __init__(self):
        self.data_min_: np.ndarray | None = None
        self.data_max_: np.ndarray | None = None
        self.scale_: np.ndarray | None = None
        self.min_: np.ndarray | None = None

    def fit(self, X: np.ndarray) -> "SafeNormalizer":
        self.data_min_ = X.min(axis=0)
        self.data_max_ = X.max(axis=0)
        denom = (self.data_max_ - self.data_min_) # the denominator
        # Avoid division by zero for constant features
        denom[denom == 0.0] = 1.0
        self.scale_ = 1.0 / denom
        self.min_ = -self.data_min_ * self.scale_
        return self

    def transform(self, X: np.ndarray) -> np.ndarray:
        assert self.scale_ is not None and self.min_ is not None
        return X * self.scale_ + self.min_

    def fit_transform(self, X: np.ndarray) -> np.ndarray:
        return self.fit(X).transform(X) # we first fit and then transform

# Distance metrics (vectorized over reference set)

def euclidean_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise Euclidean distances between rows of A (n_a,d) and B (n_b,d).
    # ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b
```

```

A2 = np.sum(A*A, axis=1, keepdims=True)
B2 = np.sum(B*B, axis=1, keepdims=True).T # we need to transpose B2
d2 = A2 + B2 - 2 * (A @ B.T) # this is the distance matrix
np.maximum(d2, 0.0, out=d2) # this is to make sure that the distance is
return np.sqrt(d2) # this is the euclidean distance matrix - of dim n_a

def manhattan_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise L1 distances."""
    # Broadcast subtraction then sum abs along features
    return np.sum(np.abs(A[:, None, :] - B[None, :, :]), axis=2) # this is t

def cosine_distance_matrix(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Pairwise cosine distance = 1 - cosine similarity."""
    A_norm = np.linalg.norm(A, axis=1, keepdims=True)
    B_norm = np.linalg.norm(B, axis=1, keepdims=True)
    # Avoid division by zero
    A_norm[A_norm == 0.0] = 1.0
    B_norm[B_norm == 0.0] = 1.0
    sim = (A @ B.T) / (A_norm * B_norm.T)
    # Numerical safety: clamp
    sim = np.clip(sim, -1.0, 1.0)
    return 1.0 - sim

class KNNClassifier:
    """Simple k-NN classifier supporting custom pairwise distance functions.
    Expects y in {0,1}. Predicts majority vote among k nearest neighbors.
    """
    def __init__(self, k: int, distance_fn: Callable[[np.ndarray, np.ndarray], np.ndarray]):
        assert k >= 1
        self.k = k
        self.distance_fn = distance_fn
        self.X_train: np.ndarray | None = None
        self.y_train: np.ndarray | None = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> "KNNClassifier":
        self.X_train = X
        self.y_train = y
        return self

    def predict_proba_positive(self, X: np.ndarray) -> np.ndarray:
        assert self.X_train is not None and self.y_train is not None
        # Compute distances from queries to all training points
        D = self.distance_fn(X, self.X_train) # shape (n_test, n_train)
        # Indices of k closest neighbors per row
        knn_idx = np.argpartition(D, kth=self.k-1, axis=1)[:, :self.k] # arg
        # For exact ordering among top-k (optional):
        # row-wise sort of the top-k slice
        row_indices = np.arange(X.shape[0])[:, None]
        knn_sorted = knn_idx[row_indices, np.argsort(D[row_indices, knn_idx])]
        # Compute fraction of positive labels among top-k
        pos_counts = np.sum(self.y_train[knn_sorted], axis=1)
        return pos_counts / self.k

    def predict(self, X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
        p = self.predict_proba_positive(X)
        return (p >= threshold).astype(int)

```

```
# Metrics

def precision_recall_f1(y_true: np.ndarray, y_pred: np.ndarray) -> Tuple[float, float, float]:
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    if precision + recall == 0:
        f1 = 0.0
    else:
        f1 = 2 * precision * recall / (precision + recall) # this is the harmonic mean
    return precision, recall, f1
```

Q2 (b): Evaluate k=31 with Euclidean distance (normalized)

We scale with train-only MinMax, fit k-NN, and compute F1 on the held-out test set.

Q2 (b): Implement a k-NN algorithm from scratch to classify the dataset. Use `k = 31` and the `euclidean` distance, and **make sure to normalize the data**. Report your model's F1 score on the test set.

```
In [7]: # Normalize train and test using train-only stats, then evaluate k=31
normalizer = SafeNormalizer().fit(X_train)
X_train_scaled = normalizer.transform(X_train)
X_test_scaled = normalizer.transform(X_test)

knn31 = KNNClassifier(k=31, distance_fn=euclidean_distance_matrix).fit(X_train_scaled)
y_pred_test = knn31.predict(X_test_scaled)
pre, rec, f1 = precision_recall_f1(y_test, y_pred_test)
print(f"k=31 Euclidean | Precision={pre:.4f} Recall={rec:.4f} F1={f1:.4f}")
```

k=31 Euclidean | Precision=1.0000 Recall=0.8810 F1=0.9367

k=31 Euclidean | Precision=1.0000 Recall=0.8810 F1=0.9367

Q2 (c): 5-fold cross-validation over k and distance metrics

We implement CV from scratch, using only NumPy. For each distance, we sweep k and record mean F1 across folds. Then we plot F1 vs k and select the best pair.

Q2 (c): Use cross-validation with 5 folds and the F1 score to tune the value of k and the distance function used (possible distance functions to use could be Euclidean distance, Manhattan distance, or cosine similarity). Make sure to use at least five values of k between 1 and 63, and try at least two distance functions. For each distance function, show a plot where the x-axis depicts the value of k and the y-axis depicts the average F1 score for that value of k during cross-validation. Which pair of parameters performed the best? Note: You may find the array split method from NumPy helpful when implementing cross-validation.

```

In [16]: # 5-fold CV implementation (train set only) and plotting
from numpy import array_split # useful for splitting the data into folds

def kfold_indices(n, n_splits, rng):
    idx = np.arange(n)
    if rng is not None:
        rs = np.random.RandomState(rng)
        rs.shuffle(idx)
    folds = array_split(idx, n_splits)
    for i in range(n_splits):
        val_idx = folds[i]
        train_idx = np.concatenate([folds[j] for j in range(n_splits) if j != i])
        yield train_idx, val_idx

k_values = [1, 3, 7, 15, 31, 47, 63]
distance_fns = {
    "euclidean": euclidean_distance_matrix,
    "manhattan": manhattan_distance_matrix,
    "cosine": cosine_distance_matrix,
}

# Normalize once using full train stats? I don't think that's a good idea -
# To avoid leakage, each fold fits scaler on fold-train only.
def cv_mean_f1_for_distance(distance_name: str, distance_fn: Callable[[np.ndarray, np.ndarray], float]):
    mean_f1_per_k: list[float] = []

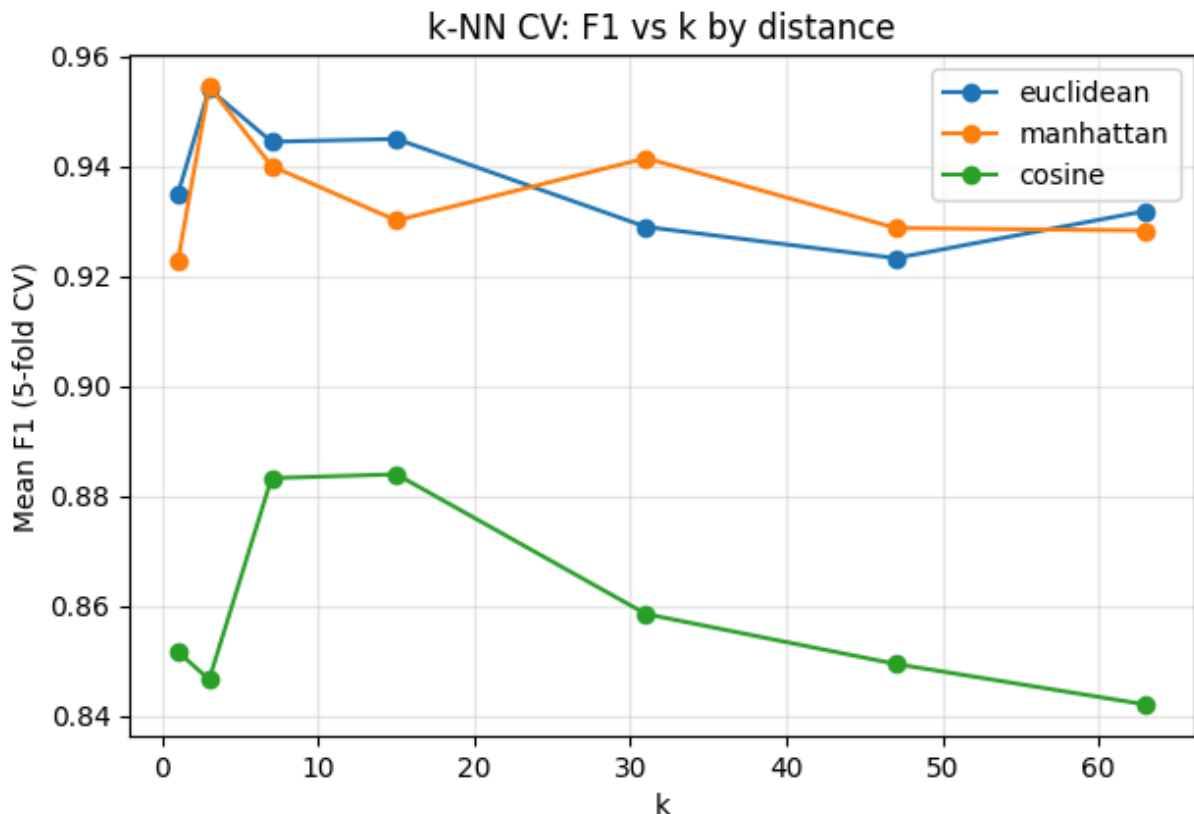
    for k in k_values:
        f1_scores: list[float] = []
        for tr_idx, va_idx in kfold_indices(len(X_train), n_splits=5, rng=42):
            X_tr, X_va = X_train[tr_idx], X_train[va_idx]
            y_tr, y_va = y_train[tr_idx], y_train[va_idx]
            # Fit scaler on fold-train only
            scaler_cv = SafeNormalizer().fit(X_tr)
            X_tr_s = scaler_cv.transform(X_tr)
            X_va_s = scaler_cv.transform(X_va)
            # Train and eval
            model = KNNClassifier(k=k, distance_fn=distance_fn).fit(X_tr_s, y_tr)
            y_va_pred = model.predict(X_va_s)
            _, _, f1_va = precision_recall_f1(y_va, y_va_pred)
            f1_scores.append(f1_va)
        mean_f1_per_k.append(float(np.mean(f1_scores)))
    return k_values, mean_f1_per_k

results = {}
plt.figure(figsize=(7,4.5))
# used chatgpt to help me plot this better
for name, fn in distance_fns.items():
    ks, mean_f1s = cv_mean_f1_for_distance(name, fn)
    results[name] = {"k": ks, "mean_f1": mean_f1s}
    plt.plot(ks, mean_f1s, marker='o', label=name)
plt.xlabel('k')
plt.ylabel('Mean F1 (5-fold CV)')
plt.title('k-NN CV: F1 vs k by distance')
plt.legend()

```

```
plt.grid(True, alpha=0.3)
plt.show()

# Find best pair
best_name, best_k, best_score = None, None, -1.0
for name, res in results.items():
    for k, s in zip(res["k"], res["mean_f1"]):
        if s > best_score:
            best_name, best_k, best_score = name, k, s
print(f"Best by CV: distance={best_name}, k={best_k}, mean F1={best_score:.4f}")
```



Best by CV: distance=manhattan, k=3, mean F1=0.9546

Q2 (d): Using the best parameters determined in part (c), report the performance of a k-NN classifier on the test set. Compare this to your model in part (b). Is it as you expected?

Awsome, so it seems that the best metrics are using K = 3 and the manhattan distance.

```
In [20]: # Refit scaler on full training set and evaluate best model on test
# Reuse `results` and `best_name`, `best_k` computed above.
best_distance_fn = distance_fns['manhattan']
best_k = 3

scaler_best = SafeNormalizer().fit(X_train)
X_train_best = scaler_best.transform(X_train)
X_test_best = scaler_best.transform(X_test)

best_model = KNNClassifier(k=best_k, distance_fn=best_distance_fn).fit(X_train_best, y_train)
y_test_pred_best = best_model.predict(X_test_best)
```

```
pre_b, rec_b, f1_b = precision_recall_f1(y_test, y_test_pred_best)
print(f"Best params on test | distance={best_name}, k={best_k} -> Precision=
# Brief comparison with part (b)
print("Compared to fixed k=31 Euclidean above.")
```

Best params on test | distance=manhattan, k=3 -> Precision=1.0000 Recall=0.9286 F1=0.9630

Compared to fixed k=31 Euclidean above.

Best parameters on the test set: distance = Manhattan, k = 3, resulting in Precision = 1.0000, Recall = 0.9286, F1 score = 0.9630.

When comparing the two models, the version with $k = 31$ and Euclidean distance exhibited very high precision but missed more malignant cases, which lowered its recall. After tuning with cross-validation, the choice of $k = 3$ and Manhattan distance performed better because a smaller k makes the classifier more sensitive to local data structures, and Manhattan distance can sometimes more effectively capture differences between samples when features vary across scales. This makes sense given that the data is imbalanced; a smaller k is more likely to be influenced by local structures. From the graph, we can see that Manhattan and Euclidean distances are quite similar, making it interesting that we achieve better performance with Manhattan distance compared to angle metrics like cosine similarity.

Q2 (e): ROC curve and AUC (from scratch)

We compute positive-class vote fractions, sweep thresholds, compute (FPR, TPR), plot ROC, and calculate AUC using trapezoidal rule. The grouping-by-vote approach is used for efficiency.

Q2 (e -> i): Using the test dataset and the same parameters as part (c), plot the ROC curve of your classifier as points connected by lines. You may implement the ROC curve computation using either of the two methods discussed in the HW PDF

```
In [25]: # --- Elegant grouping ROC for k-NN (approach 2) ---

def roc_by_grouping(y_true: np.ndarray, proba_pos: np.ndarray, k: int) -> tu
    """
    Build ROC by grouping points with exactly i positive neighbors (i in {0.
    We sweep from i=k down to 0, accumulating TP and FP.
    """
    y_true = np.asarray(y_true)
    proba_pos = np.asarray(proba_pos)

    # Recover integer neighbor counts; guard tiny float noise.
    # proba_pos should be exactly {0/k, 1/k, ..., k/k} for majority-vote k-NN.
    kpos = np rint(proba_pos * k).astype(int)
    kpos = np.clip(kpos, 0, k)

    P = int(np.sum(y_true == 1))
```

```

N = int(np.sum(y_true == 0))

# Start at (0,0): predicting nothing positive
tprs = [0.0]
fprs = [0.0]
tp = 0
fp = 0

# Add groups from i = k down to 0 - here k is 3
for i in range(k, -1, -1):
    mask = (kpos == i)
    if np.any(mask):
        tp += int(np.sum(y_true[mask] == 1))
        fp += int(np.sum(y_true[mask] == 0))
        tprs.append(tp / P if P else 0.0)
        fprs.append(fp / N if N else 0.0)

# Ensure we end at (1,1) (in case of any numerical weirdness)
tprs[-1] = 1.0 if P else tprs[-1]
fprs[-1] = 1.0 if N else fprs[-1]

return np.array(fprs), np.array(tprs)

def auc_trapezoid(x: np.ndarray, y: np.ndarray) -> float:
    """Simple trapezoidal AUC (assumes x is sorted ascending)."""
    area = 0.0
    for i in range(1, len(x)):
        dx = x[i] - x[i-1] # this is the marginal x
        area += dx * (y[i] + y[i-1]) / 2.0 # this is the area of the trapezoid
    return float(area)

proba_pos = best_model.predict_proba_positive(X_test_best)
fpr_g, tpr_g = roc_by_grouping(y_test, proba_pos, 3) # our best k is 3

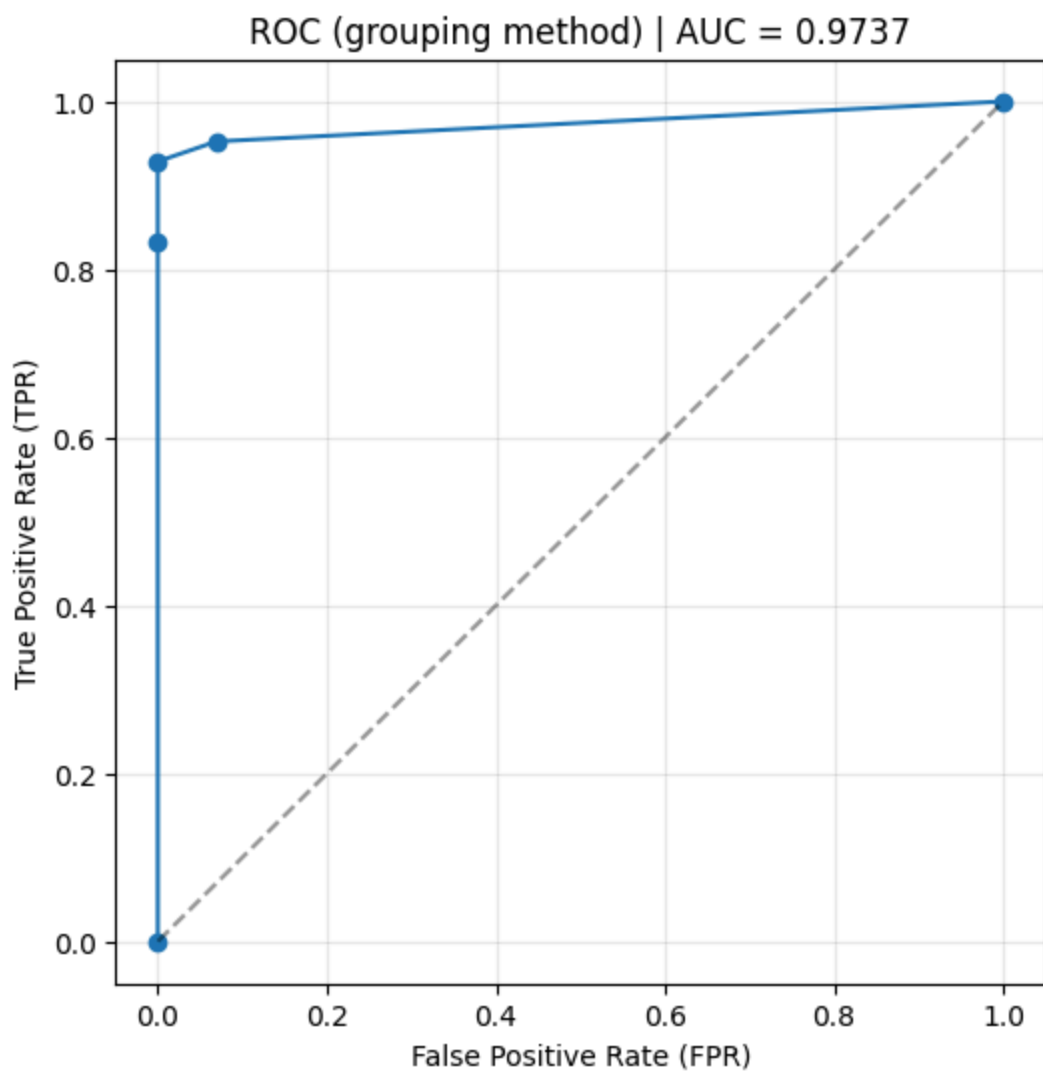
# Sort by FPR just in case equal-FPR plateaus appear out of order
order = np.argsort(fpr_g)
fpr_g, tpr_g = fpr_g[order], tpr_g[order]

auc_g = auc_trapezoid(fpr_g, tpr_g)

plt.figure(figsize=(6,6))
plt.plot(fpr_g, tpr_g, marker='o', lw=1.5)
plt.plot([0,1], [0,1], 'k--', alpha=0.4)
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'ROC (grouping method) | AUC = {auc_g:.4f}')
plt.grid(True, alpha=0.3)
plt.show()

print(f"AUC (grouping method, from scratch) = {auc_g:.6f}")

```



AUC (grouping method, from scratch) = 0.973710

CS 671 Fall 2025

HW1, Q3

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: Brian Mason and Cursor/GPT5 -- though I do want to emphasize that the code even if LLM generated was based on my reasoning and understanding of the problem and LLMs were simply used to help me write the code in a more efficient manner.

Agreement 2: I understand that if I struggle with this assignment that I will reevaluate whether this is the correct class for me to take. I understand that the homework only gets harder.

Setup

```
In [1]: !pip install --upgrade treefarms
```

Requirement already satisfied: treefarms in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (0.2.4)

Requirement already satisfied: setuptools in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (75.8.0)

Requirement already satisfied: wheel in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.45.1)

Requirement already satisfied: attrs in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (24.3.0)

Requirement already satisfied: packaging>=20.9 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (24.2)

Requirement already satisfied: pandas in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.3.2)

Requirement already satisfied: scikit-learn in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (1.7.2)

Requirement already satisfied: sortedcontainers in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.4.0)

Requirement already satisfied: gmpy2 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (2.2.1)

Requirement already satisfied: matplotlib in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (3.10.6)

Requirement already satisfied: tqdm in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (4.67.1)

Requirement already satisfied: timbertrek in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.1.7)

Requirement already satisfied: editables==0.2 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from treefarms) (0.2)

Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (1.3.2)

Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (4.60.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (1.4.9)

Requirement already satisfied: numpy>=1.23 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (2.0.1)

Requirement already satisfied: pillow>=8 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (3.2.4)

Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from matplotlib->treefarms) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pandas->treefarms) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pandas->treefarms) (2025.1)

Requirement already satisfied: scipy>=1.8.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (1.15.3)

Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (1.5.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from scikit-learn->treefarms) (3.6.0)

Requirement already satisfied: ipython in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from timbertrek->treefarms) (8.30.0)

Requirement already satisfied: six>=1.5 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib->treefarms) (1.16.0)

Requirement already satisfied: decorator in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (5.1.1)

Requirement already satisfied: jedi>=0.16 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.19.2)

Requirement already satisfied: matplotlib-inline in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.1.6)

Requirement already satisfied: prompt-toolkit<3.1.0,>=3.0.41 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (3.0.43)

Requirement already satisfied: pygments>=2.4.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (2.15.1)

Requirement already satisfied: stack-data in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (0.2.0)

Requirement already satisfied: traitlets>=5.13.0 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (5.14.3)

Requirement already satisfied: exceptiongroup in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (1.2.0)

Requirement already satisfied: typing-extensions>=4.6 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (4.12.2)

Requirement already satisfied: pexpect>4.3 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from ipython->timbertrek->treefarms) (4.8.0)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from jedi>=0.16->ipython->timbertrek->treefarms) (0.8.4)

Requirement already satisfied: ptyprocess>=0.5 in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from pexpect>4.3->ipython->timbertrek->treefarms) (0.7.0)

Requirement already satisfied: wcwidth in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from prompt-toolkit<3.1.0,>=3.0.41->ipython->timbertrek->treefarms) (0.2.5)

Requirement already satisfied: executing in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (0.8.3)

Requirement already satisfied: asttokens in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (2.0.5)

Requirement already satisfied: pure-eval in /opt/anaconda3/envs/deeplearning/lib/python3.10/site-packages (from stack-data->ipython->timbertrek->treefarms) (0.2.2)

```
In [2]: import pandas as pd
import os
from treefarms import TREEFARMS
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import f1_score
from sympy import to_dnf, simplify_logic
```

```
In [3]: def to_dnf_string(leaf) -> str:
        """Converts a leaf of a tree to a DNF expression
        for the path of variables to reach the leaf.
        """
        rules = leaf['rules']
        dnf = []
        for feature, rule in rules.items():
            if rule['type'] == 'Categorical':
                if rule['positive']:
                    dnf.append(f'({feature})')
                else:
                    dnf.append(f'(~{feature})')
        return ' & '.join(dnf)

def convert_to_dnf(tree):
    """Converts a tree to a DNF of its leaves.
    e.g. the tree (right splits are 1, left are 0)
        x3
      //  \
    x1    1
   //  \
  0     x2
   //  \
  1     0

    becomes the DNF

    (x1 & ~x2) | x3.
    """
    leaves = tree.__groups__()

    #otherwise, iterate through leaves
    pos_dnfs = []
    for leaf in leaves:
        if leaf['prediction'] == 1:
            pos_dnfs.append(to_dnf_string(leaf))

    # cover case where tree simplifies to True or False (like a stump, or a
    # tree with many leaves that all predict the same thing)
    if len(pos_dnfs) == 0: # if no leaf can ever predict positive:
        return False

    return to_dnf(' | '.join(pos_dnfs))

def get_simplified_dnf(tree):
    """Converts a tree to a simplified DNF of its leaves.
    """
    pos_dnf = convert_to_dnf(tree)
    simple_pos_dnf = simplify_logic(pos_dnf, form='dnf', force=True)
    return simple_pos_dnf

def print_dnf(dnf, features):
    """Print a DNF using the given feature names.
```

This is needed because sympy representations of DNFs use feature indices instead of their names. For example, the expression $(a \ \& \ \sim b) \mid c$ will be represented as $(\text{feature}_0 \ \& \ \sim \text{feature}_1) \mid \text{feature}_2$.

Use this to print DNF expressions in part d).

Parameters

```
dnf: A sympy expression representing a DNF.
features: A list of strings representing the feature names.
"""
dnf_str = str(dnf)
features = list(features)
for i in reversed(range(0, len(features))):
    dnf_str = dnf_str.replace(f'feature_{i}', features[i])
print(dnf_str)
```

```
In [4]: df = pd.read_csv('compas_hw1_q3.csv')
target = 'recidivate-within-two-years:1'
X = df.drop(target, axis=1)
y = df[target]
h = df.columns

# split the data into 70% train, 20% val, 10% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=42)

print('train size:', X_train.shape[0]/X.shape[0])
print('val size:', X_val.shape[0]/X.shape[0])
print('test size:', X_test.shape[0]/X.shape[0])
```

```
train size: 0.6998696974084262
val size: 0.2000868683943825
test size: 0.10004343419719125
```

```
In [5]: # train TREEFARMS model
config = {
    "regularization": 0.01, # regularization penalizes the tree with more leaves
    "rashomon_bound_multiplier": 0.05, # rashomon bound multiplier indicates how much more complex the tree can be
    "depth_budget": 6
}

model = TREEFARMS(config)

model.fit(X_train, y_train)

models = [model[i] for i in range(model.get_tree_count())]
```

```

null
Finding Optimal Objective...
treefarm reported successful execution
training completed. Number of trees in the Rashomon set: 1603
{
  "false": {
    "false": {
      "complexity": 0.009999999776482582,
      "loss": 0.02027306519448757,
      "name": "recidivate-within-two-years:1",
      "prediction": 1
    },
    "feature": 7,
    "name": "juvenile-crimes:=0",
    "reference": 1,
    "relation": "==",
    "true": {
      "complexity": 0.009999999776482582,
      "loss": 0.21679767966270447,
      "name": "recidivate-within-two-years:1",
      "prediction": 0
    },
    "type": "integral"
  },
  "feature": 11,
  "model_objective": 0.3754695951938629,
  "name": "priors:>3",
  "reference": 1,
  "relation": "==",
  "true": {
    "complexity": 0.009999999776482582,
    "loss": 0.10839883983135223,
    "name": "recidivate-within-two-years:1",
    "prediction": 1
  },
  "type": "integral"
}
{
  "false": {
    "complexity": 0.009999999776482582,
    "loss": 0.0376499779522419,
    "name": "recidivate-within-two-years:1",
    "prediction": 1
  },
  "feature": 7,
  "model_objective": 0.3754695951938629,
  "name": "juvenile-crimes:=0",
  "reference": 1,
  "relation": "==",
  "true": {
    "false": {
      "complexity": 0.009999999776482582,
      "loss": 0.21679767966270447,
      "name": "recidivate-within-two-years:1",
      "prediction": 0
    },

```

```

    "feature": 11,
    "name": "priors:>3",
    "reference": 1,
    "relation": "==",
    "true": {
      "complexity": 0.009999999776482582,
      "loss": 0.09102192521095276,
      "name": "recidivate-within-two-years:1",
      "prediction": 1
    },
    "type": "integral"
  },
  "type": "integral"
}

```

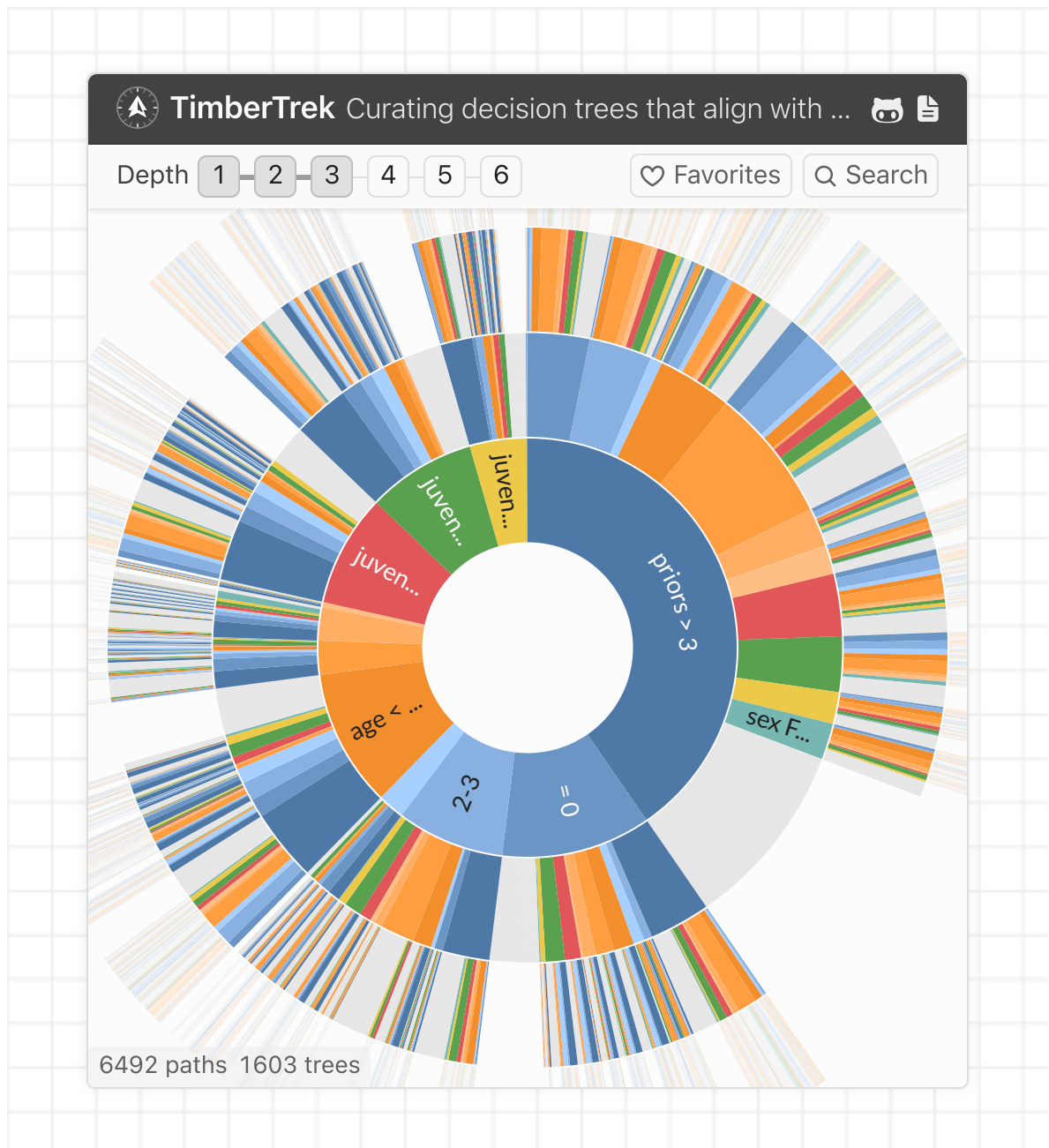
This is a TimberTrek representation of the Rashomon set of decision trees. Tree splits at different depths are represented in concentric rings, with rings further from the center representing deeper levels of the trees. Grey bars are individual models in the Rashomon set -- try clicking around to find models that you like, and play around with the search function to get rid of models using certain features!

In [6]: `model.visualize()`

```

Generating trie from 1603 trees: 100%|██████████| 1603/1603 [00:00<00:00, 62
944.99it/s]
Generating decision paths from 1603 trees: 100%|██████████| 1603/1603 [00:03
<00:00, 522.39it/s]

```



a) Exploring the Rashomon set

The Rashomon set fitting code is provided for you in the setup.

- i. Compute and store the number of leaves of each model in the Rashomon set, the train accuracy of each model, the validation accuracy of each model, and the validation f1 score of each model.

ii. Report the minimum and maximum of each item you computed in part i.

Q3 (a): Rashomon set model statistics

We compute leaves, train/val accuracy, and validation F1 for each model in the set, then report min/max.

```
In [7]: # Helper to compute accuracy

def accuracy(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return float((y_true == y_pred).mean())

# Collect per-model stats
stats = []
for m in models:
    # number of leaves (len of positive + negative groups)
    leaves = len(m.__groups__)
    # predictions
    yhat_tr = m.predict(X_train)
    yhat_va = m.predict(X_val)
    # metrics
    tr_acc = accuracy(y_train, yhat_tr)
    va_acc = accuracy(y_val, yhat_va)
    va_f1 = f1_score(y_val, yhat_va)
    stats.append({
        'leaves': leaves,
        'train_acc': tr_acc,
        'val_acc': va_acc,
        'val_f1': va_f1,
        'model': m,
    })

# Summary
leaves_list = [s['leaves'] for s in stats]
tr_list = [s['train_acc'] for s in stats]
va_list = [s['val_acc'] for s in stats]
va_f1_list = [s['val_f1'] for s in stats]

print(f"Leaves:      min={min(leaves_list)} max={max(leaves_list)}")
print(f"Train Acc:  min={min(tr_list):.4f} max={max(tr_list):.4f}")
print(f"Val Acc:     min={min(va_list):.4f} max={max(va_list):.4f}")
print(f"Val F1:      min={min(va_f1_list):.4f} max={max(va_f1_list):.4f}")
```

```
Leaves:      min=2 max=6
Train Acc:   min=0.6345 max=0.6721
Val Acc:     min=0.6353 max=0.6831
Val F1:      min=0.5607 max=0.6751
```

Q3 (b): Demographic Parity Difference for sex:Female

We compute DPD for each model, summarize min/max, and briefly interpret.

Interpretation: A DPD near 0 indicates similar average positive predictions across sexes.

A positive DPD means the model predicts positives more often when `sex:Female=0` (males) than `sex:Female=1` (females); negative means the reverse. The spread between min/max indicates variability in how models rely on sex.

b) Calculating Demographic Parity Difference

i. Fill in the provided function for computing demographic parity difference.

```
In [8]: def demographic_parity_difference(model, X, sensitive_feature):
        """Calculate the demographic parity difference of a model.

        Parameters
        -----
        model: A model implementing the predict function.
        X: A pandas dataframe of features.
        sensitive_feature: The name of the sensitive feature.

        Returns
        -----
        (float) The demographic parity difference.
        """
        # Predict on X
        y_hat = model.predict(X)
        s = X[sensitive_feature].values.astype(int)
        idx0 = (s == 0)
        idx1 = (s == 1)
        # Handle empty group edge cases gracefully
        p0 = y_hat[idx0].mean() if idx0.any() else 0.0
        p1 = y_hat[idx1].mean() if idx1.any() else 0.0
        return float(p0 - p1)
```

ii. Calculate the demographic parity difference of all of the models in the Rashomon set for the `sex:Female` feature. Report the minimum and maximum demographic parity difference inside the Rashomon set.

```
In [9]: # Compute DPD on validation set for sex:Female
        sensitive = 'sex:Female'
        for s in stats:
            s['val_dpd'] = demographic_parity_difference(s['model'], X_val, sensitive)
```

```
val_dpd_list = [s['val_dpd'] for s in stats]
print(f"DPD (sex:Female): min={min(val_dpd_list):.4f} max={max(val_dpd_list)}
```

```
DPD (sex:Female): min=0.1408 max=0.4461
```

iii. Interpret the minimum and maximum demographic parity differences inside the Rashomon set. What do the results imply about the variety of ways that the models in the Rashomon set decide if someone is likely to recidivate based on their sex?

The demographic parity difference among Rashomon-set models ranges from about 0.14 to 0.45. This means that, depending on the model, males are predicted to be between 14% and 45% more likely to recidivate than females on average. In other words, even though all these models are near-optimal in terms of accuracy, they vary widely in how strongly they use sex in making predictions. Some models exhibit relatively low disparities and treat males and females similarly, while others rely much more heavily on sex, leading to large differences in predictions. This highlights that the Rashomon set contains models with very different fairness properties, and shows why accuracy alone isn't enough to choose a model — we need to consider fairness metrics like demographic parity difference when deciding which model to deploy.

Q3 (c): Distributions and filtering

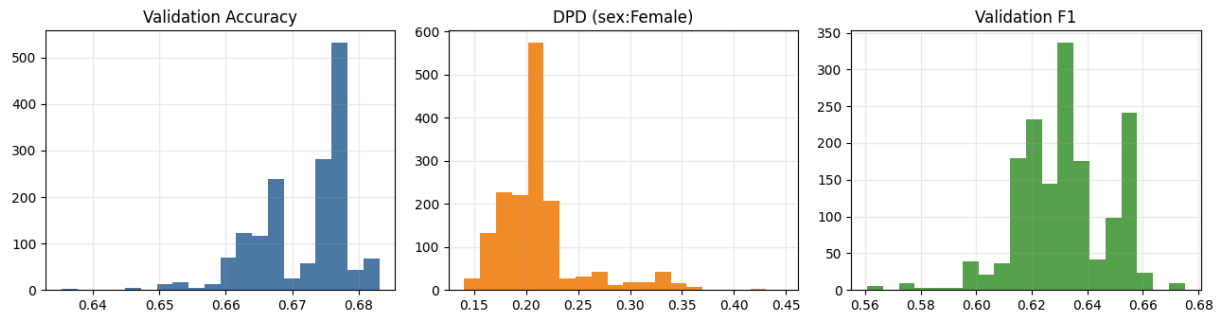
```
In [10]: # Histograms (20 bins)
vals_acc = [s['val_acc'] for s in stats]
vals_f1 = [s['val_f1'] for s in stats]
vals_dpd = [s['val_dpd'] for s in stats]

plt.figure(figsize=(13,3.5))
plt.subplot(1,3,1); plt.hist(vals_acc, bins=20, color='#4e79a7'); plt.title(
plt.subplot(1,3,2); plt.hist(vals_dpd, bins=20, color='#f28e2b'); plt.title(
plt.subplot(1,3,3); plt.hist(vals_f1, bins=20, color='#59a14f'); plt.title(
plt.tight_layout(); plt.show()

# Filters
filter1 = [s for s in stats if s['val_acc'] > 0.67]
print(f"After val_acc > 0.67: {len(filter1)} models")

filter2 = [s for s in filter1 if s['val_dpd'] < 0.2]
print(f"After DPD < 0.2: {len(filter2)} models")

filter3 = [s for s in filter2 if s['val_f1'] > 0.65]
print(f"After val_f1 > 0.65: {len(filter3)} models")
```



After `val_acc > 0.67`: 1004 models

After `DPD < 0.2`: 103 models

After `val_f1 > 0.65`: 34 models

c) Filtering the Rashomon set

i. Plot histograms, with 20 bins, of the validation accuracies, demographic parity differences w.r.t `sex:Female`, and validation f1 scores in the Rashomon set. We will inspect these histograms to choose cut points to filter the Rashomon set.

ii. Filter the Rashomon set down to models with validation accuracies > 0.67 . How many models are you left with?

After `val_acc > 0.67`: 1004 models

iii. Filter again down to models with validation demographic parity differences < 0.2 . How many models are you left with?

After `DPD < 0.2`: 103 models

iv. Filter once more to models with a validation f1-score > 0.65 . How many models are you left with?

After `val_f1 > 0.65`: 34 models

Q3 (d): Inspect DNFs of remaining models

```
In [11]: # to print just the DNFs
         ## Build set of simplified DNFs for the final filtered models
```

```

# remaining = [s['model'] for s in filter3]

# dnfs = []
# for m in remaining:
#     dnf = get_simplified_dnf(m)
#     dnfs.append(str(dnf))

# unique_dnfs = sorted(set(dnfs))
# print(f"Unique simplified DNFs: {len(unique_dnfs)}")
# for i, d in enumerate(unique_dnfs, 1):
#     print(f"[{i}] {d}")

# let's try to print both the DNFs and the Features

features = list(X.columns)

remaining = [{ 'model': s['model'], 'val_f1': s['val_f1'], 'val_dpd': s['val_

def name_dnf(dnf_expr):
    d = str(dnf_expr)
    for i in reversed(range(len(features))):
        d = d.replace(f'feature_{i}', features[i])
    return d

named_to_stats = {}
for r in remaining:
    dnf = get_simplified_dnf(r['model'])
    d_named = name_dnf(dnf)
    named_to_stats.setdefault(d_named, []).append((r['val_f1'], r['val_dpd']

print(f"Unique simplified DNFs (named): {len(named_to_stats)}")
# Order by best F1 within each DNF (desc)
for i, (d, lst) in enumerate(
    sorted(named_to_stats.items(), key=lambda kv: max(x[0] for x in kv[1]),
):
    f1s = [x[0] for x in lst]
    dpds = [x[1] for x in lst]
    print(f"[{i}] best F1={max(f1s):.4f} mean F1={np.mean(f1s):.4f} mean D
    print(f"      {d}")

```

```

Unique simplified DNFs (named): 9
[1] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    age:<21 | (priors:2-3 & age:<26) | (priors:>3 & ~priors:2-3) | (~priors:
2-3 & ~juvenile-crimes:=0)
[2] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    age:<21 | priors:>3 | (priors:2-3 & age:<26) | (~priors:2-3 & ~juvenile-
crimes:=0)
[3] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=6
    (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) | (priors:>3 & ~priors:
2-3) | (~priors:2-3 & ~juvenile-crimes:=0)
[4] best F1=0.6544 mean F1=0.6544 mean DPD=0.1982 count=2
    priors:>3 | (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) | (~priors:
2-3 & ~juvenile-crimes:=0)
[5] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=12
    age:<21 | priors:>3 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26)
[6] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    age:<21 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (priors:>3 & ~p
riors:2-3)
[7] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    priors:>3 | ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (age:<21 & ~p
riors:2-3)
[8] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=4
    priors:>3 | ~juvenile-crimes:=0 | (age:<21 & age:<26) | (priors:2-3 & ag
e:<26)
[9] best F1=0.6534 mean F1=0.6534 mean DPD=0.1981 count=2
    ~juvenile-crimes:=0 | (priors:2-3 & age:<26) | (age:<21 & ~priors:2-3) |
(priors:>3 & ~priors:2-3)

```

d) Inspecting the remaining models

i. Use the provided helper code to examine the logical forms of the remaining models. How many unique simplified DNFs did we filter down to?

Only 9!

ii. Among the unique simplified DNF forms that you have remaining, use logical equivalence between expressions along with what you know about the features to identify how many of these are truly unique. (Hint: for example, you can't have `priors:>3` and `priors:=0`!)

Using the dataset semantics:

- **Priors buckets are one-hot:** exactly one of { `priors:=0` , `priors:=1` , `priors:2-3` , `priors:>3` } is true.
 $\Rightarrow$ `(priors:>3 & ~priors:2-3)  $\equiv$  priors:>3`.

- **Age thresholds are nested:** `age:<21 ⇒ age:<26` .
- `~juvenile-crimes:=0` means “has ≥1 juvenile offense”.

and simplifying we get 2 unique DNFs:

- **[1]–[4]** become the same formula after removing redundancies
(e.g., the union of `(age:<21 & ~priors:2-3)` and `(priors:2-3 & age:<26)` is effectively `age:<21 v (priors:2-3 & age:<26)` ; and `(priors:>3 & ~priors:2-3)` is just `priors:>3` , etc.).
- **[5]–[9]** collapse to a second formula because each contains the unconditional disjunct `~juvenile-crimes:=0` (predict positive for any juvenile record), plus the same two “priors/age” patterns as above.

Hence the 9 DNFs reduce to **2 truly distinct logical functions**:

- **Class A (covers [1]–[4]):**

`[\text{age:<21} ;;\lor;; \text{priors:>3} ;;\lor;; (\text{priors:2-3} \land \text{age:<26})
;;\lor;; (\lnot\text{priors:2-3} \land \lnot\text{juvenile-crimes:=0})]`

(i.e., uses juvenile history *conditionally* on priors not being 2–3.)

- **Class B (covers [5]–[9]):**

`[\text{age:<21} ;;\lor;; \text{priors:>3} ;;\lor;; (\text{priors:2-3} \land \text{age:<26})
;;\lor;; \lnot\text{juvenile-crimes:=0}]`

(i.e., triggers on *any* juvenile record regardless of priors.)

iii. The few remaining models are very similar to each other! Which one of them would you deploy? There are no wrong answers to this question.

Between these two models:

- **Class A ([1]–[4]):** best/mean F1 = 0.6544, mean DPD ≈ 0.1982.
- **Class B ([5]–[9]):** best/mean F1 = 0.6534, mean DPD ≈ 0.1981.

Accuracy and fairness metrics are almost identical, so the tie-breaker is the **logic**:

- **Class A** only uses juvenile history in context (requires “not priors:2–3,” with other priors disjuncts already explicit).

- **Class B** fires on *any* juvenile record (`~juvenile-crimes:=0`), regardless of priors—this is broader and more punitive.

Both families achieve nearly the same performance, but Class A is more measured: it doesn't penalize juvenile history in isolation, and instead considers it alongside adult priors. This aligns with a cautious, humane stance—acknowledging risk where multiple indicators agree (very young age, substantial adult priors, or a specific priors-and-age pattern), while avoiding blanket inferences from any juvenile record alone. Given the similar metrics, choosing the rule set that's less sweeping and more contextual feels fairer to the people affected.

This is a very cool problem, tysm for giving me the opportunity to work on it!